COS 214 - Practical 2

Team members

- Keagan van Biljon - 23594412
- Thakasa Ngcobo - 25556526
- Leul T Tamiru - 25270665

Task 1 – Event concept and completed architecture

1.1 Event Definition — PixelForge Gaming Convention

PixelForge is a two-day indoor gaming convention held across a single large venue. Attendees move between hands-on demo areas, competitive tournament stages, and support services throughout the day, while a central control desk pushes live updates (queue changes, safety alerts, schedule shifts) to whichever areas need to react.

The venue is organised into three kinds of operational area:

- Expo Zone — houses hands-on hardware and game demo areas and the main ticketing/check-in point.
- Arena Zone — houses the competitive tournament stages where scheduled matches run throughout the day.
- Community Zone — houses support services: food vendors, medical staffing, and inter-venue shuttle services.

Each zone is itself made up of individual operational units:

- DemoStation — a hands-on booth where attendees try upcoming games; tracks how many stations are currently free.
- TournamentArena — a competitive stage running a scheduled match; tracks which match is currently live.
- TicketDesk — the check-in/admission point; can stop or resume admitting attendees.
- FoodVendor — a food stall; tracks remaining stock and can suspend service.
- MedicalStation — a first-aid point; stays operational through most disruptions.
- Shuttle — an inter-venue transport service; can change its route when an area closes.

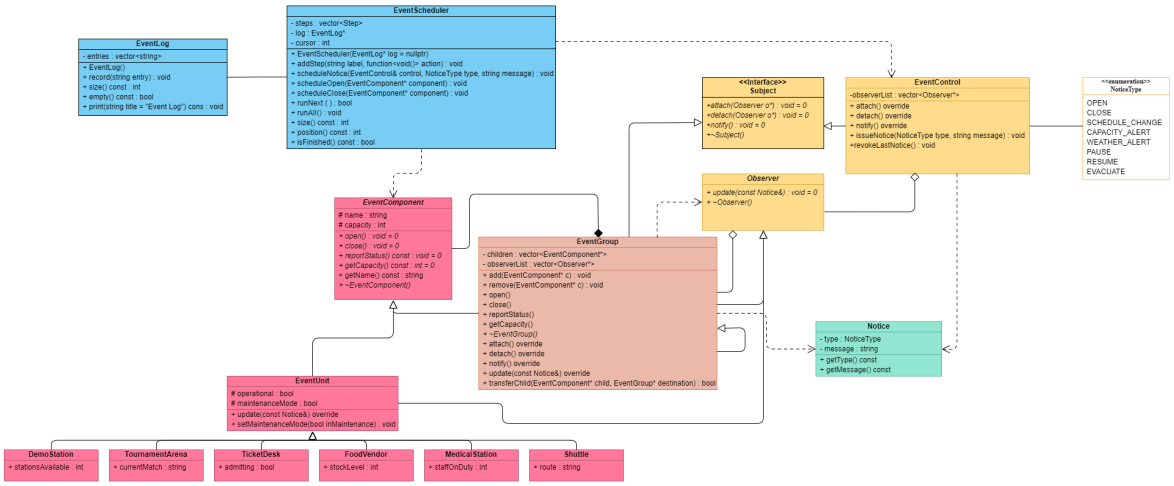
This gives five concrete leaf types plus a sixth for extra behavioural range, and three distinct grouping contexts, all nested beneath a single root PixelForge event object; satisfying the minimum Composite depth once zones, sub-areas, and units are instantiated together (see the object diagram in Task 2.3).

1.2 GoF Participant Mapping

Class	Composite role	Observer role
EventComponent	Component (abstract)	—
EventGroup	Composite	ConcreteObserver and ConcreteSubject
EventUnit	Leaf (abstract)	ConcreteObserver (base)
DemoStation, TournamentArena, TicketDesk, FoodVendor, MedicalStation, Shuttle	Leaf	ConcreteObserver (each overrides update() distinctly)
Subject	—	Subject (abstract)
Observer	—	Observer (abstract)
EventControl	—	ConcreteSubject
Notice	—	the state/data passed in a notification

EventGroup participates in both patterns. As a Composite, it owns its children, the parts that make up the venue's part-whole tree. As an Observer, it receives notices pushed down from a parent zone or from EventControl. As a Subject, it maintains its own observerList of units below it and re-broadcasts a notice further down once it has processed it. These are three separate collaborations happening through the same object, each backed by a distinct attribute (children vs. the inherited observerList) not one pattern wearing another's clothes.

1.3 Design Rationale



(for a better view, see class_uml.png)

1.4 Design Rationale

(a) Composite is a part-whole tree. A PixelForge root contains zones; zones contain sub-areas or units directly; and operations like getCapacity() and reportStatus() only make sense as an aggregate of the parts beneath a node. Deleting or querying a zone must affect everything nested inside it — the defining property of composition rather than a flat collection.

(b) Observer is required, not hard-coded calls. EventControl has no knowledge of how many zones exist, what they're called, or what they contain. It only knows it has a list of Observer*. New zones or units can register at runtime without EventControl (or any zone above them) being modified. A hard-coded call chain would need to know every concrete class in advance, which the practical explicitly forbids.

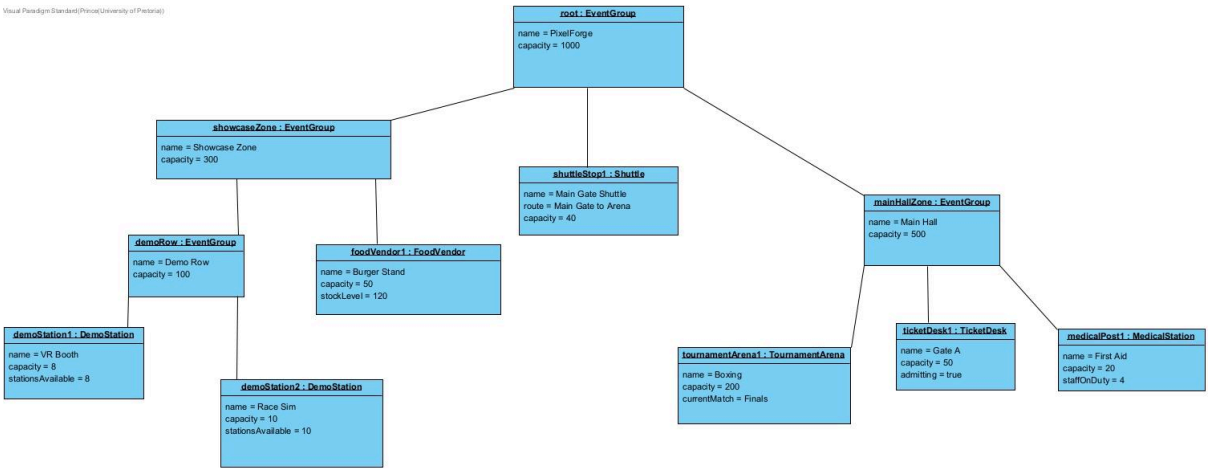
(c) Ownership of event components. EventGroup owns its children via children : vector<EventComponent*> and deletes them in its destructor — composition, not aggregation. An EventComponent never outlives the group that owns it unless explicitly detached and reassigned during a runtime reorganisation (Task 4.2).

(d) Ownership of observer references. Observer pointers stored in observerList are non-owning. A Subject never deletes an observer; it only forgets about it via detach(). This is necessary because the same object (e.g. a TicketDesk) is owned by exactly one EventGroup in the Composite tree, but may be registered as an observer with a different EventGroup, or with EventControl directly — ownership and notification are independent relationships, so neither can carry deletion responsibility for the other.

(e) Being both Subject and Observer is not a misuse of either pattern. EventGroup is not reusing one pattern's machinery to fake the other; it holds two separate role-specific data structures (children for ownership, observerList for notification) and fulfils two separate contracts (EventComponent's and Observer's pure virtual operations). Being notified from above and notifying below are legitimately different responsibilities that happen to fall on the same object because, structurally, a zone genuinely sits in the middle of both the containment tree and the notification chain.

Task 2

2.3 Draw an object diagram showing the concrete object instances and the ownership tree.



2.4 Demonstrate destruction of the root and explain why the complete owned subtree is released exactly once.

Because ~EventComponent() is virtual, deleting the root runs ~EventGroup(), which loops through its children vector and deletes each pointer. This chains down through ~EventUnit() to the concrete leaf destructors. Every object is deleted exactly once because it only ever exists in one children vector at a time.

Task 3

3.5 Push or Pull.

Push, and the code makes the choice unambiguous: Observer::update() takes const Notice& directly, and every notify() (EventControl::notify(), EventGroup::notify()) loops its observer list calling obs->update(*currentNotice) — the full state (type + message) is handed over in the call itself. Nothing in update() ever calls back into the Subject to ask "what changed" — every leaf's update() override branches purely on notice.getType()/getMessage().

Trade-off:

Push means leaves never need to hold a pointer back to their Subject just to receive notifications; EventUnit::update() is self-contained, and EventGroup::update() can rebroadcast by just forwarding the same Notice& it received. That's less coupling and simpler leaves. The cost is that a Notice is all a leaf ever gets. If some leaf ever needed a piece of Subject-side state that isn't packaged into type/message (say, the Subject's live capacity at the moment of the alert), push alone can't deliver it; the leaf would have to separately call a query method like getCapacity() on something it already holds a reference to rather than getting it "for free" through the notification itself.

Task 4

4.1 Define at least five event rules that cause concrete units to react differently to the same notice.

Leaf	Extra field	Reaction to notices
TicketDesk	admitting : bool	OPEN: admitting = true; CLOSE: admitting = false; CAPACITY_ALERT: admitting = false; RESUME: admitting = true;
TournamentArena	currentMatch : std::string	WEATHER_ALERT or PAUSE: operational = false; · RESUME: operational = true;
Shuttle	route : std::string	WEATHER_ALERT or EVACUATE: route = "Evacuation Loop";
MedicalStation	staffOnDuty : int	EVACUATE: staffOnDuty += 2;
FoodVendor	stockLevel : int	CAPACITY_ALERT: if (stockLevel < 10) stockLevel = 0; else stockLevel -= 10;
DemoStation	stationsAvailable : int	CLOSE: stationsAvailable = 0; CAPACITY_ALERT: if (stationsAvailable < 1) stationsAvailable = 0; else stationsAvailable -= 1;

4.2

Using transferChild() removes the child from the current group before adding it to the destination. It also updates both Composite ownership (using remove/add) and Observer registration (using detach/attach) in a single sequence.

4.3

The inherited capacity field is used as an alert threshold. When a CAPACITY_ALERT is received, the group checks if its getCapacity() (the sum of its children) is greater than or equal to this threshold. If yes, it passes on the notice via notify(); if no, it doesn't react.

4.4 Briefly explain why each feature could be added without turning one class into a god object.

Feature 1 = Maintenance Mode (EventUnit)
Any leaf can be pulled offline via setMaintenanceMode(true): getCapacity() drops to 0 and update() ignores every notice except EVACUATE (safety always gets through). It fits naturally; a real festival takes stalls or stations offline for repairs without shutting the whole

zone down. It stays small on purpose: two bools and a few ifs inside methods that already existed (getCapacity(), update()) — no new class, no new collaboration path, so it can't grow into a god object; it's just a gate on behaviour that's already there.

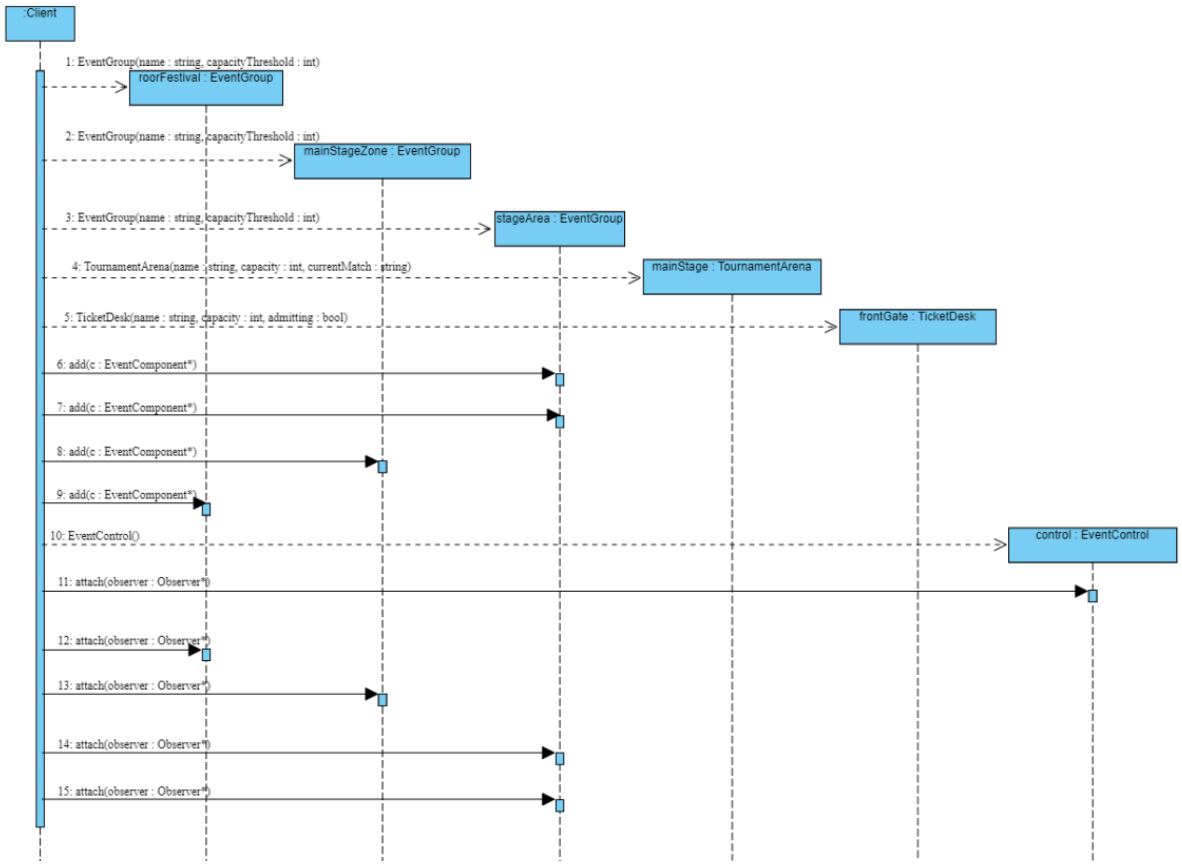
Feature 2 = Notice Revocation (EventControl::revokeLastNotice())
Reverses the last notice where a sensible reversal exists (PAUSE→RESUME, CLOSE→OPEN, WEATHER_ALERT→RESUME), and is a safe no-op otherwise (e.g. after EVACUATE). It's contained entirely inside EventControl and reuses the existing issueNotice()/cascade machinery rather than duplicating it — it doesn't touch Composite or any leaf, so responsibility for "what a notice means" stays exactly where it already lived.

Feature 3 = EventScheduler
A queue of labelled steps (Step{label, action}) stepped through one at a time. It turns the Task 8 demo checklist into a reusable script instead of a wall of sequential calls in main(). Crucially, EventScheduler itself has zero domain knowledge — it doesn't know what a Notice is or what EventGroup::transferChild does; it only knows how to hold and run callables. All the actual EventFlow logic lives in the lambdas supplied from main.cpp. That's what keeps it from becoming a god object: it can't accumulate event-domain responsibility because it structurally never touches event-domain types beyond the two convenience wrappers (scheduleNotice, scheduleOpen/scheduleClose) built on the already-frozen EventControl/EventComponent interfaces.

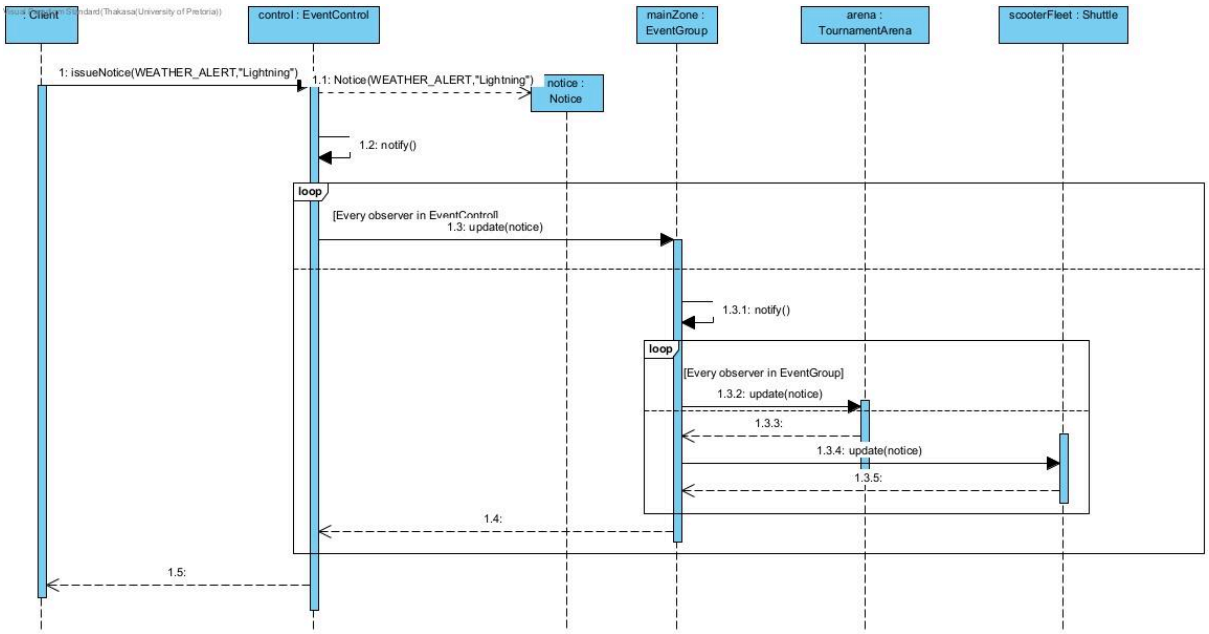
Feature 4 = EventLog
Even narrower: records strings, prints strings. It doesn't know EventScheduler exists beyond receiving record(label) calls, and knows nothing about Notice, EventComponent, or anything else. A one-method-does-the-work class is about as far from a god object as you can get.

Task 5 - Sequence Diagram Portfolio

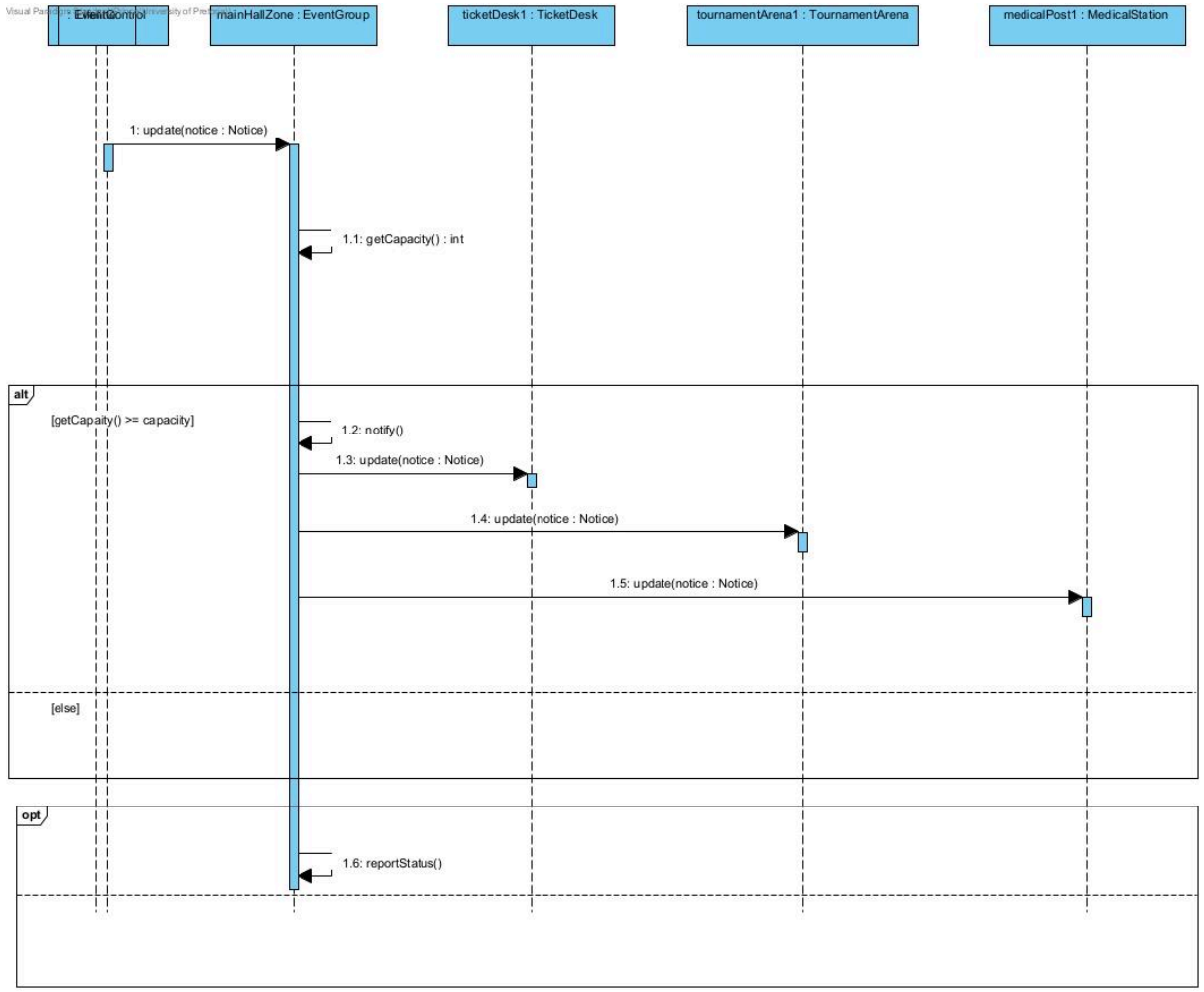
5.1 - Building and registering part of the event



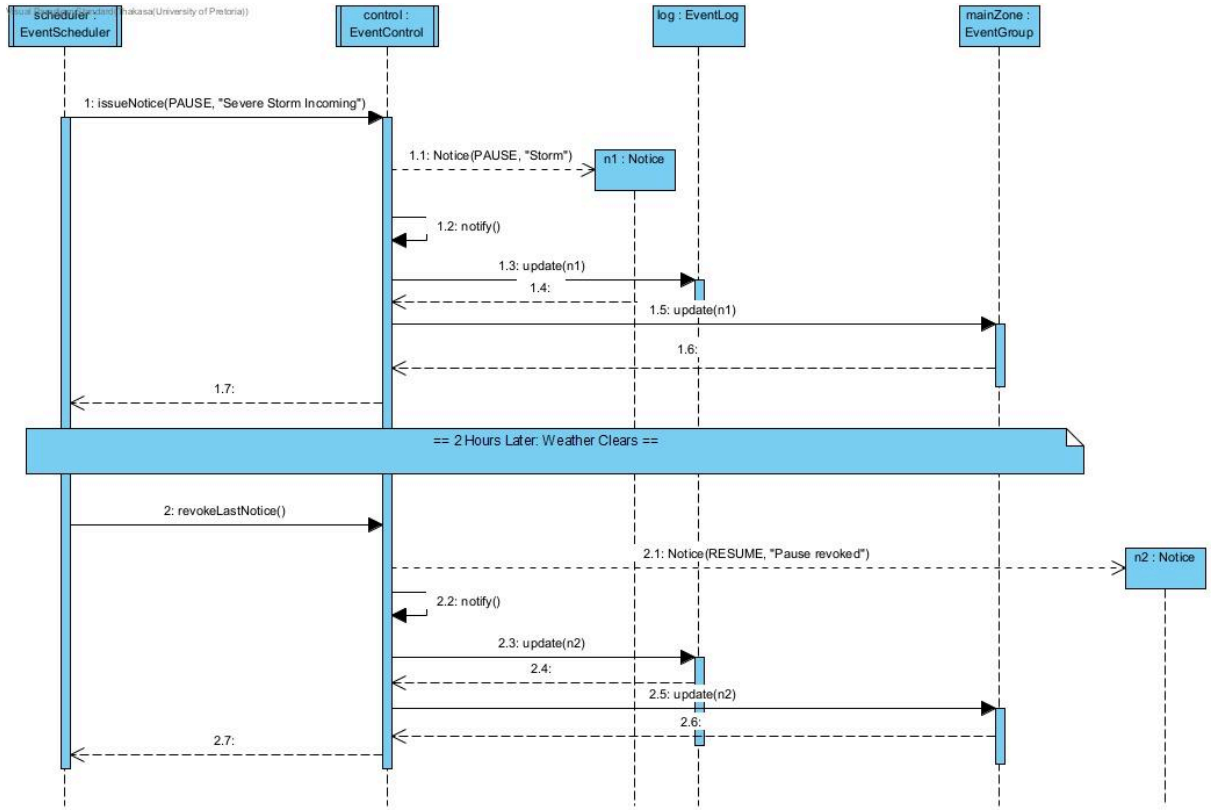
5.2 -Cascading event notification



5.3 - Conditional event response and Composite behaviour



5.4 - Your signature event scenario



Task 6

Task 7

https://github.com/Keagan-VB/COS214_Prac3.git

7.4 GitHub Workflow Reflection

We split the work along the two design patterns plus integration: one teammate owned the Composite side (EventComponent, EventUnit, the six concrete leaves, and EventGroup), one owned the Observer side (Subject, Observer, Notice, NoticeType, and EventControl), and the third owned Task 1's write-up, main.cpp, the two original features (EventScheduler and EventLog), and project scaffolding (Makefile, README, Doxyfile).

This split was possible because on day one, before any implementation began, we agreed and froze four interface headers together: EventComponent.h, Subject.h, Observer.h, and Notice.h/NoticeType.h. Method signatures only, no bodies. Once those were fixed, each of us could build our own files against an interface rather than against a teammate's unfinished code, so nobody was blocked mid-week waiting on someone else.

We worked on three long-lived feature branches - `composite`, `observer`, and `main` - each tracking one person's files. Three pull requests merged the pattern branches into ‘main’ once each side was functionally complete and self-tested.

Three commits/activities that show real collaboration rather than a single end-of-project dump:
1. “Added Observer and Subject interface headers” - e.g. the commit on day one that added the Observer and Subject interface headers.
2. “Task 3: Observer” - e.g. a commit on the `observer` branch implementing task 3.
3. “Created the testing main” - e.g. the `main` branch commit when the completed testing main was added in.

Branching this way meant integration was a single, deliberate step rather than continuous friction: nobody touched EventGroup.cpp except the Composite owner, nobody touched Subject.h/Observer.h/EventControl.cpp except the Observer owner, so the two pattern branches never produced merge conflicts against each other - the only merges that mattered were each pattern branch into `main`, which is also where the completed testing main.cpp was actually wired to the real classes for the first time.